

CO1 – Assignment 2 (AT2)

Machine Learning — Selected Questions (Python Implementation) | Reg. No: 192412179

Q3. Find-S Algorithm — Play Tennis Dataset (5 Marks)

Ex	Outlook	Temperature	Humidity	Wind	Play Tennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rainy	Mild	High	Weak	Yes
5	Rainy	Cool	Normal	Weak	Yes
6	Rainy	Cool	Normal	Strong	No
7	Overcast	Mild	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No

Python Code:

```
def find_s(data):
    hypothesis = None
    for row in data:
        *attrs, label = row
        if label == 'Yes':
            # use positive examples only
            if hypothesis is None:
                hypothesis = attrs.copy()
            else:
                for i in range(len(hypothesis)):
                    if hypothesis[i] != attrs[i]:
                        hypothesis[i] = '?' # generalize mismatched attribute
                print("Update ->", hypothesis)
    return hypothesis

tennis = [
    ["Sunny", "Hot", "High", "Weak", "No"],
    ["Sunny", "Hot", "High", "Strong", "No"],
    ["Overcast", "Hot", "High", "Weak", "Yes"],
    ["Rainy", "Mild", "High", "Weak", "Yes"],
    ["Rainy", "Cool", "Normal", "Weak", "Yes"],
    ["Rainy", "Cool", "Normal", "Strong", "No"],
    ["Overcast", "Mild", "Normal", "Strong", "Yes"],
    ["Sunny", "Mild", "High", "Weak", "No"],
]
h = find_s(tennis)
print("Final hypothesis:", h)
```

Output:

```
Update -> ['Overcast', 'Hot', 'High', 'Weak']
Update -> ['?', '?', 'High', 'Weak']
Update -> ['?', '?', '?', 'Weak']
Update -> ['?', '?', '?', '?']
Final hypothesis: ['?', '?', '?', '?']
```

Final Hypothesis $h = \langle ?, ?, ?, ? \rangle$

Q4. Candidate Elimination Algorithm — Enjoy Sport Dataset (5 Marks)

Ex	Weather	Temperature	Humidity	Wind	Enjoy Sport
1	Sunny	Warm	Normal	Weak	Yes
2	Sunny	Warm	High	Strong	Yes
3	Rainy	Cold	High	Weak	No
4	Sunny	Cold	Normal	Weak	Yes
5	Rainy	Warm	Normal	Strong	No

Python Code:

```
def more_general(h1, h2):
    return all(x == '?' or x == y for x, y in zip(h1, h2))

def candidate_elimination(data):
    n = len(data[0]) - 1
    S = ['0'] * n          # most specific boundary
    G = [['?'] * n]        # most general boundary
    for row in data:
        *attrs, label = row
        if label == 'Yes':
            G = [g for g in G if more_general(g, attrs)]
            if S == ['0'] * n:
                S = attrs.copy()
            else:
                for i in range(n):
                    if S[i] != attrs[i]:
                        S[i] = '?'
        else:
            new_G = []
            for g in G:
                if more_general(g, attrs):
                    for i in range(n):
                        if g[i] == '?' and S[i] not in ('?', attrs[i]):
                            new_h = g.copy()
                            new_h[i] = S[i]
                            new_G.append(new_h)
                else:
                    new_G.append(g)
            G = new_G
    print("After", attrs, label, "-> S =", S, " G =", G)
    return S, G

sport = [
    ["Sunny", "Warm", "Normal", "Weak", "Yes"],
    ["Sunny", "Warm", "High", "Strong", "Yes"],
    ["Rainy", "Cold", "High", "Weak", "No"],
    ["Sunny", "Cold", "Normal", "Weak", "Yes"],
    ["Rainy", "Warm", "Normal", "Strong", "No"],
]
S, G = candidate_elimination(sport)
print("Final S:", S, " Final G:", G)
```

Output:

```
After ['Sunny', 'Warm', 'Normal', 'Weak'] Yes -> S=['Sunny', 'Warm', 'Normal', 'Weak'] G=[['?', '?', '?', '?']]
After ['Sunny', 'Warm', 'High', 'Strong'] Yes -> S=['Sunny', 'Warm', '?', '?'] G=[['?', '?', '?', '?']]
After ['Rainy', 'Cold', 'High', 'Weak'] No -> S=['Sunny', 'Warm', '?', '?'] G=[['Sunny', '?', '?', '?'], ['?', 'Warm',
After ['Sunny', 'Cold', 'Normal', 'Weak'] Yes -> S=['Sunny', '?', '?', '?'] G=[['Sunny', '?', '?', '?']]
After ['Rainy', 'Warm', 'Normal', 'Strong'] No -> S=['Sunny', '?', '?', '?'] G=[['Sunny', '?', '?', '?']]
Final S: ['Sunny', '?', '?', '?'] Final G: [['Sunny', '?', '?', '?']]
```

Final Version Space: $S = G = \langle \text{Sunny}, ?, ?, ? \rangle$

Q7. Find-S Algorithm — Loan Approval Dataset (5 Marks)

Ex	Employment	Salary	Credit Score	Loan History	Loan Approved
1	Employed	High	Good	Clean	Yes
2	Employed	Medium	Good	Clean	Yes
3	Unemployed	Low	Poor	Bad	No
4	Employed	Medium	Excellent	Clean	Yes

Python Code:

```
loan = [
    ["Employed", "High", "Good", "Clean", "Yes"],
    ["Employed", "Medium", "Good", "Clean", "Yes"],
    ["Unemployed", "Low", "Poor", "Bad", "No"],
    ["Employed", "Medium", "Excellent", "Clean", "Yes"],
]
h = find_s(loan)          # same find_s() function as Q3
print("Final hypothesis:", h)
```

Output:

```
Update -> ['Employed', 'High', 'Good', 'Clean']
Update -> ['Employed', '?', 'Good', 'Clean']
Update -> ['Employed', '?', '?', 'Clean']
Final hypothesis: ['Employed', '?', '?', 'Clean']
```

Final Hypothesis $h = \langle \text{Employed}, ?, ?, \text{Clean} \rangle$

Q9. Candidate Elimination Algorithm — Healthy Lifestyle Dataset (5 Marks)

Ex	Diet	Exercise	Sleep	Stress Level	Healthy Lifestyle
1	Balanced	Regular	Adequate	Low	Yes
2	Balanced	Regular	Poor	Medium	Yes
3	Junk	None	Poor	High	No
4	Balanced	Moderate	Adequate	Low	Yes
5	Junk	Regular	Adequate	High	No

Python Code:

```
health = [
    ["Balanced", "Regular", "Adequate", "Low", "Yes"],
    ["Balanced", "Regular", "Poor", "Medium", "Yes"],
    ["Junk", "None", "Poor", "High", "No"],
    ["Balanced", "Moderate", "Adequate", "Low", "Yes"],
    ["Junk", "Regular", "Adequate", "High", "No"],
]
S, G = candidate_elimination(health)  # same function as Q4
print("Final S:", S, " Final G:", G)
```

Output:

Final Version Space: $S = G = \langle \text{Balanced}, ?, ?, ? \rangle$

(c) K-Means vs Hierarchical Clustering:

Aspect	K-Means	Hierarchical Clustering
No. of clusters	Must specify K in advance	Not required; cut dendrogram
Approach	Partition-based, centroid updates	Builds tree via merge/split
Cost	$\sim O(n)$ per iteration	$\sim O(n^2)$ or higher
Cluster shape	Best for spherical clusters	Captures nested/irregular shapes
Sensitivity	Sensitive to init centroids	Deterministic (no random init)